

A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline

Ph.D. Student's Activity Report



Federico Bruzzone 

Computer Science Department, Università degli Studi di Milano

Supervisor: Prof. **Walter Cazzola**

Email: federico.bruzzone@unimi.it

Website: federicobruzzone.github.io

22 June 2026

Optimizing compilers

Since the dawn of computing, compiler optimizations have long bridged high-level abstractions and efficient machine code [1–6].

Optimizing compilers [7, 8] use analysis passes to guide transformations [9–11] that enhance performance without altering observable behavior [12–14]—modern architectures (e.g., x86-64, Aarch64) make this an interplay of interdependent passes [9].

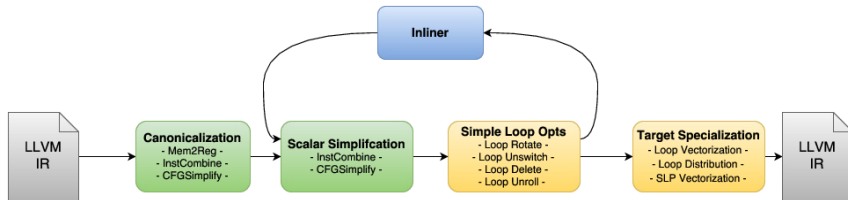
LLVM [15] exemplifies this complexity, serving as the standard backend for C/C++, Rust [16], Julia [17], and via **MLIR** [18]—for *machine learning* (ML) and *deep learning* (DL) stacks [19, 20] that rely on ML/DL optimizing compilers [21–24] (e.g., NVCC¹, OpenXLA [25], TVM [26]) and automotive pipelines.²

¹developer.nvidia.com/cuda-llvm-compiler

²Tesla Full Self-Driving v14.3: stats.tessie.com/versions/2026.2.9.6

LLVM Optimization Pipeline

LLVM is a modular SSA-based compilation framework [27–29]. Its opt tool applies sequences of analysis and transformation passes, forming the core optimization pipeline.



It comprises parameterized nested managers (**module**, **function**, **cgscc**, **loop**, **loop-mssa**), each scoped to a specific IR granularity.

We need better optimization evaluation!

Per-pass impact analysis is critical for compiler engineers and end-users [30–32] to:

- ➡ address *undecidable* [33–35] phase ordering challenges [36–38],
- ➡ improve selection heuristics and ML-based auto-tuning [39–45],
- ➡ guide new optimization design, and
- ➡ understand pass–hardware interactions [34].

However, existing work evaluates LLVM optimizations in isolation or focuses mainly on execution time [46], overlooking multi-dimensional trade-offs—a pass improving runtime may increase binary size or hurt cache locality.

Moreover, the debate over hand-written GPU kernels vs. ML-compiler-optimized kernels remains open [47].³

³[octoml-optimizes-apache-tvm-for-apples-m1-beats-core-ml-4-by-29](#)

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM -O3 pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**⁴ similar to the methodology of Chen *et al.* [51].

RQ₁. *What is the marginal performance impact of individual passes within the LLVM -O3 optimization pipeline?*

RQ₂. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ₃. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ₄. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

⁴E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, cycles/instructions for ILP, and energy consumption [48–50] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM -O3 pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**⁴ similar to the methodology of Chen *et al.* [51].

RQ₁. *What is the marginal performance impact of individual passes within the LLVM -O3 optimization pipeline?*

RQ₂. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ₃. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ₄. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

⁴E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, cycles/instructions for ILP, and energy consumption [48–50] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM -O3 pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**⁴ similar to the methodology of Chen *et al.* [51].

RQ₁. *What is the marginal performance impact of individual passes within the LLVM -O3 optimization pipeline?*

RQ₂. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ₃. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ₄. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

⁴E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, cycles/instructions for ILP, and energy consumption [48–50] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM -O3 pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**⁴ similar to the methodology of Chen *et al.* [51].

RQ₁. *What is the marginal performance impact of individual passes within the LLVM -O3 optimization pipeline?*

RQ₂. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ₃. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ₄. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

⁴E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, cycles/instructions for ILP, and energy consumption [48–50] via RAPL.

A Study to Rule Them All (150+ studies at ICSE'26 A*)

We address this gap with a multi-dimensional, systematic empirical study of the LLVM -O3 pipeline, measuring not only **runtime** but also **compile time**, **binary size**, and a broad set of **HW performance counters**⁴ similar to the methodology of Chen *et al.* [51].

RQ₁. *What is the marginal performance impact of individual passes within the LLVM -O3 optimization pipeline?*

RQ₂. *To what extent do optimization passes trade off execution speed, compilation overhead, and binary footprint?*

RQ₃. *How do IR-level transformations affect the evolution of HW performance counters across optimization pipelines?*

RQ₄. *What fraction of achievable speedup is lost to intra-pipeline interference, and which pass drives the loss?*

⁴E.g., those exposed by Model-Specific Registers (MSRs) on x86 CPUs: cache misses, cycles/instructions for ILP, and energy consumption [48–50] via RAPL.

Experimental setup

All experiments ran on Intel Core i9-12900KF:

- ☞ Alder Lake (8P+8E cores, 5.2 GHz max);
- ☞ 128 GB RAM;
- ☞ Fedora Linux 43 (kernel 6.19);
- ☞ Cache hierarchy: 640 KiB **L1-D**, 768 KiB **L1-I**, 14 MiB **L2**, 30 MiB **L3**.

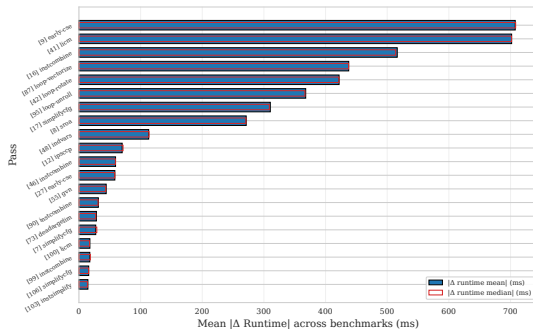
We used LLVM 21.1.8 compiled in release, with no assertions, *profile-guided optimizations* (PGO) disabled, studying the -O3 pipeline.

Runtime uses hyperfine (3 warmup, 15 timed runs), counters use perf stat (2 warmup, 10 timed) in a separate batch—10 runs suffice for CI95 below 1% on all counter metrics.

Cumulative speedup, normalised gain, and top-20 passes



$$G_b^i = (S_b^i - 1) / (S_b^* - 1) \in [0, 1]$$



- ☞ Pareto-skewed distribution: the top decile accounts for the majority of impact.
- ☞ The ranking is dominated by early-cse, licm, instcombine, loop-vectorize, loop-rotate, loop-unroll, simplifycfg, sroa, indvars.

2mm Linear-Algebra Kernel from PolyBench [52]⁵

A double matrix-matrix multiplication (GEMM) in $O(N^3)$ time.

```
/* D = a*A*B*C + b*D */
for (i = 0; i < NI; i++)
  for (j = 0; j < NJ; j++)
    for (k = 0; k < NK; k++)
      t[i][j] += a*A[i][k]*B[k][j];
for (i = 0; i < NI; i++)
  for (j = 0; j < NL; j++) {
    D[i][j] *= b;
    for (k = 0; k < NJ; k++)
      D[i][j] += t[i][k]*C[k][j];
  }
```

Practical utility of the kernel:

- 👉 **Data Reuse:** CPU/GPU spatial and temporal cache locality.
- 👉 **Pipeline Parallelism:** CPU/GPU SIMD, *instruction level* parallelism (ILP), and superscalar pipelined execution.
- 👉 **Deep Learning:** the basic block during inference (and training) in neural networks is 2mm.
- 👉 **Features:** *affine* perfect (first) and imperfect (second) loop nests suitable for polyhedral optimizations.

⁵github.com/MatthiasJReisinger/PolyBenchC-4.2.1/polybench.pdf

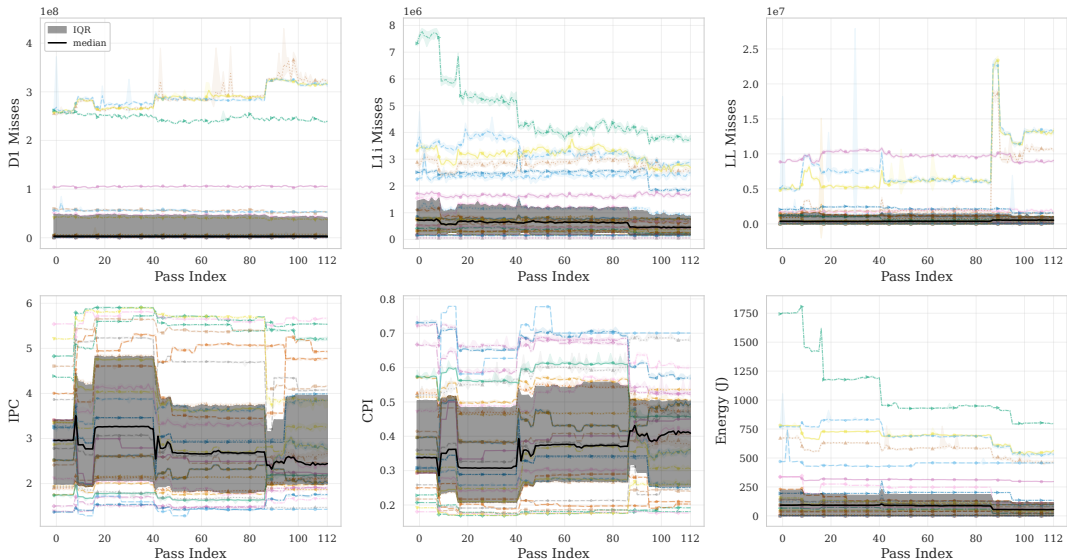
HW-counter signatures at the top-10 passes (2mm)

The blue-colored boxes are **improvements** over the baseline (-00), while red-colored are **regressions**.

	2mm									
sroa [8]	6	4	12	14	-0	14	-13	0	2	
early-cse [9]	4	7	12	-10	-3	-8	9	0	2	
ipscpp [12]	-0	1	11	-14	-3	-12	13	0	-3	
instcombine [16]	0	7	12	4	-2	5	-5	0	-3	
simplifycfg [17]	3	9	13	4	-2	5	-5	0	2	
licm [41]	37	45	28	14	70	-33	49	0	56	
loop-rotate [42]	0	11	16	-3	4	-7	8	0	5	
indvars [48]	2	6	14	-7	4	-10	12	0	3	
loop-vectorize [87]	1	1	12	-7	3	-10	11	-0	4	
loop-unroll [95]	-11	-10	-10	-19	-30	15	-13	0	-26	
	d1_misses	l1_i_misses	l1_misses	instructions	cycles	ipc	cpi	branch_misses	energy-joules	

- 👉 The branch predictor is not waiting: branch_misses are 0 everywhere! Thanks to prefetching due to constant stride.
- 👉 licm is introducing lots of regressions, but recovered later.
- 👉 loop-unroll decreased d1, l1_i, and l1_d counters. The energy consumption is also decreasing.
- 👉 loop-vectorize is not so significant, but it is still worth mentioning.

HW counters aggregated across the PolyBench suite



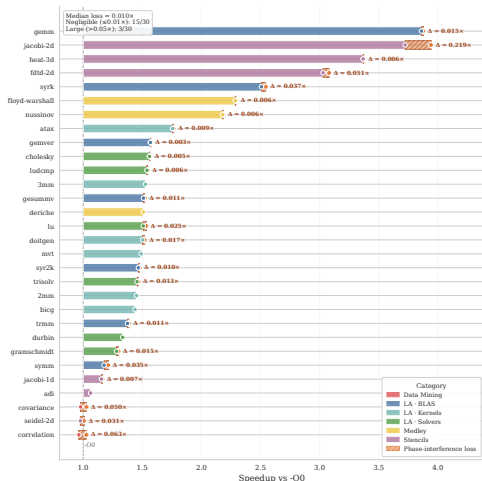
Bounded phase-interference loss

We **define** a novel optimistic additive bound:

$$S_{\text{opt}} = \sum_i \max(\Delta S_i, 0),$$

$$L = \frac{S_{\text{opt}} - [S_{\text{actual}}]_0^{S_{\text{opt}}}}{S_{\text{opt}}},$$

where $[x]_0^{S_{\text{opt}}} = \min(\max(x, 0), S_{\text{opt}})$ and $S_{\text{actual}} = S_{\text{final}} - 1$.



Thank you!



`federicobruzzone.github.io`

References I

- [1] A. P. Ershov. On Programming of Arithmetic Operations. *Communications of the ACM*, 1(8):3–6, 1958.
- [2] J. Heller. Sequencing aspects of multiprogramming. *J. ACM*, 8(3):426–439, 1961.
- [3] W. M. McKeeman. Peephole optimization. *Commun. ACM*, 8(7):443–444, 1965.
- [4] C. W. Gear. High speed compilation of efficient object code. *Commun. ACM*, 8(8):483–488, 1965.
- [5] Edward S. Lowry and C. W. Medlock. Object Code Optimization. *Communications of the ACM*, 12(1):13–22, January 1969.
- [6] Vincent A. Busam and Donald E. Englund. Optimization of expressions in fortran. *Communications of the ACM*, 12(12):666–674, 1969.
- [7] Frances E. Allen. Program optimization. In Mark Halpern and Christopher Shaw, editors, *Annual Review of Automatic Programming*, volume 5, pages 239–307. Pergamon Press, 1966.
- [8] William Allan Wulf. The design of an optimizing compiler. 1973.
- [9] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, November 2022.
- [10] Ken Kennedy and John R. Allen. *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*. Morgan Kaufmann Publishers Inc., October 2001.
- [11] R. Paige. Future directions in program transformations. *SIGPLAN Not.*, 32(1):94–98, 1997.
- [12] David F. Bacon, Susan L. Graham, and Oliver J. Sharp. Compiler Transformations for High-Performance Computing. *ACM Computing Surveys*, 26(4):345–420, December 1994.

References II

- [13] Mike Dodds, Mark Batty, and Alexey Gotsman. Compositional verification of compiler optimisations on relaxed memory. In *Programming Languages and Systems*, pages 1027–1055. Springer, 2018.
- [14] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, USA, second edition, 2006.
- [15] Chris Lattner and Vikram Adve. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. In Michael D. Smith, editor, *Proceedings of the 2nd International Symposium on Code Generation and Optimization (CGO’04)*, pages 75–86, San José, CA, USA, March 2004. IEEE.
- [16] Nicholas D. Matsakis and Felix S. Klock. The Rust Language. *ACM SIGAda Letters*, 34(3):103–104, October 2014.
- [17] Jeff Bezanson, Stefan Karpinski, Viral B Shah, and Alan Edelman. Julia: A fast dynamic language for technical computing. *arXiv:1209.5145*, 2012.
- [18] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: Scaling compiler infrastructure for domain specific computation. In *CGO ’21*, pages 2–14. IEEE, 2021.
- [19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Adv. Neural Inf. Process. Syst.*, 32, 2019.
- [20] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: a system for Large-Scale machine learning. In *OSDI ’16*, pages 265–283, 2016.

References III

- [21] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. The deep learning compiler: A comprehensive survey. *IEEE Trans. Parallel Distrib. Syst.*, 32(3):708–727, 2020.
- [22] Yu Xing, Jian Weng, Yushun Wang, Lingzhi Sui, Yi Shan, and Yu Wang. An in-depth comparison of compilers for deep neural networks on hardware. In *ICISS ’19*, pages 1–8, 2019.
- [23] Ruizhe Zhao, Shuanglong Liu, Ho-Cheung Ng, Erwei Wang, James J. Davis, Xinyu Niu, Xiwei Wang, Huifeng Shi, George A. Constantinides, Peter Y. K. Cheung, and Wayne Luk. Hardware compilation of deep neural networks: An overview. In *ASAP ’18*, pages 1–8, 2018.
- [24] Hsin-I Cindy Liu, Marius Brehler, Mahesh Ravishankar, Nicolas Vasilache, Ben Vanik, and Stella Laurenzo. Tinyiree: An ml execution environment for embedded systems from compilation to deployment. *IEEE Micro*, 42(5):9–16, 2022.
- [25] Amit Sabne. Xla: Compiling machine learning for peak performance, 2020.
- [26] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Tvm: end-to-end optimization stack for deep learning. *arXiv:1802.04799*, 2018.
- [27] Barry K Rosen, Mark N Wegman, and F Kenneth Zadeck. Global value numbers and redundant computations. In *Proc. POPL ’88*, pages 12–27, 1988.
- [28] Bowen Alpern, Mark N Wegman, and F Kenneth Zadeck. Detecting equality of variables in programs. In *Proc. POPL ’88*, pages 1–11, 1988.

References IV

- [29] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. An Efficient Method of Computing Static Single Assignment Form. In *Proceedings of the 16th Symposium on Principles of Programming Languages (POPL '89)*, pages 25–35, Austin, TX, USA, January 1989. ACM.
- [30] Lucian Popescu and Nuno P. Lopes. Exploiting undefined behavior in c/c++ programs for optimization: A study on the performance impact. *Proc. ACM Program. Lang.*, 9(PLDI), 2025.
- [31] Melina Demertzi, Murali Annavaram, and Mary Hall. Analyzing the effects of compiler optimizations on application reliability. In *IISWC '11*, pages 184–193. IEEE, 2011.
- [32] John Cavazos, Grigori Fursin, Felix Agakov, Edwin Bonilla, Michael FP O'Boyle, and Olivier Temam. Rapidly selecting good compiler optimizations using performance counters. In *CGO '07*, pages 185–197. IEEE, 2007.
- [33] Amir H. Ashouri, Andrea Bignoli, Gianluca Palermo, Cristina Silvano, Sameer Kulkarni, and John Cavazos. Micomp: Mitigating the compiler phase-ordering problem using optimization sub-sequences and machine learning. *ACM Trans. Archit. Code Optim.*, 14(3), 2017.
- [34] Jack W. Davidson, Gary S. Tyson, David B. Whalley, and Prasad A. Kulkarni. Evaluating heuristic optimization phase order search algorithms. In *CGO '07*, pages 157–169, 2007.
- [35] Steven R. Vegdahl. Phase coupling and constant generation in an optimizing microcode compiler. In *Proc. MICRO '82*, page 125–133. IEEE Press, 1982.
- [36] Sid-Ahmed-Ali Touati and Denis Barthou. On the decidability of phase ordering problem in optimizing compilation. In *Proc. CF '06*, page 147–156. ACM, 2006.

References V

- [37] Stefan M Freudenberger and John C Ruttenberg. Phase ordering of register allocation and instruction scheduling. In *Intl. Workshop on Code Generation, Dagstuhl*, pages 146–170. Springer, 1992.
- [38] Jiayu Zhao, Chunwei Xia, and Zheng Wang. Leveraging compilation statistics for compiler phase ordering. In *IPDPS '25*, pages 533–545. IEEE, 2025.
- [39] Haolin Pan, Yuanyu Wei, Mingjie Xing, Yanjun Wu, and Chen Zhao. Towards efficient compiler auto-tuning: Leveraging synergistic search spaces. In *CGO '25*, page 614–627. ACM, 2025.
- [40] Mingxuan Zhu, Dan Hao, and Junjie Chen. Compiler autotuning through multiple-phase learning. *ACM Trans. Softw. Eng. Methodol.*, 33(4), 2024.
- [41] Amir H. Ashouri, William Killian, John Cavazos, Gianluca Palermo, and Cristina Silvano. A survey on compiler autotuning using machine learning. *ACM Comput. Surv.*, 51(5), 2018.
- [42] Ananta Tiwari, Chun Chen, Jacqueline Chame, Mary Hall, and Jeffrey K Hollingsworth. A scalable auto-tuning framework for compiler optimization. In *IPDPS '09*, pages 1–12. IEEE, 2009.
- [43] Zhelong Pan and Rudolf Eigenmann. Peak—a fast and effective performance tuning system via compiler optimization orchestration. *ACM Trans. Program. Lang. Syst.*, 30(3), 2008.
- [44] Zhelong Pan and R. Eigenmann. Fast and effective orchestration of compiler optimizations for automatic performance tuning. In *CGO '06*, pages 12 pp.–332, 2006.
- [45] Prasad Kulkarni, Stephen Hines, Jason Hiser, David Whalley, Jack Davidson, and Douglas Jones. Fast searches for effective optimization phase sequences. *SIGPLAN Not.*, 39(6):171–182, 2004.

References VI

- [46] Juan Carlos de la Torre, Patricia Ruiz, Bernabé Dorronsoro, and Pedro L. Galindo. Analyzing the influence of llvm code optimization passes on software performance. In *IPMU '18*, pages 272–283. Springer, 2018.
- [47] Shoumik Palkar, James Thomas, Deepak Narayanan, Pratiksha Thaker, Rahul Palamuttam, Parimajan Negi, Anil Shanbhag, Malte Schwarzkopf, Holger Pirk, Saman Amarasinghe, Samuel Madden, and Matei Zaharia. Evaluating end-to-end optimization for data analytics applications in weld. *Proc. VLDB Endow.*, 11(9):1002–1015, 2018.
- [48] Nicolas van Kempen, Hyuk-Je Kwon, Dung Tuan Nguyen, and Emery D Berger. It’s not easy being green: On the energy efficiency of programming languages. In *ASE '25*, pages 1553–1565. IEEE, 2025.
- [49] Sung Une Lee, Niroshinie Fernando, Kevin Lee, and Jean-Guy Schneider. A survey of energy concerns for software engineering. *J. Syst. Softw.*, 210:111944, 2024.
- [50] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy Efficiency Across Programming Languages. In Marjan Mernik and Bernhard Rumpe, editors, *Proceedings of the 10th International Conference on Software Language Engineering (SLE'17)*, pages 256–267, Vancouver, BC, Canada, October 2017. ACM.
- [51] Yang Chen, Shuangde Fang, Yuanjie Huang, Lieven Eeckhout, Grigori Fursin, Olivier Temam, and Chengyong Wu. Deconstructing iterative optimization. *ACM Trans. Archit. Code Optim.*, 9(3), 2012.
- [52] Louis-Noël Pouchet et al. Polybench: The polyhedral benchmark suite. 437:1–1, 2012.